Demonstration scenario

A ten to fifteen minute walkthrough of the tool on the reference corpus. Every number below is produced by the code on the captures in tests/corpus/; nothing is narrated that the tool does not show. The window was driven through this scenario before writing it down, and what it shows is recorded in docs/GUI_VERIFICATION.md; the same investigation without the window is docs/demo_cli.md.

Preparation

Environment:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

Files the walkthrough opens:

- tests/corpus/reference_export/corpus_capture_01.normalized.json - the first capture, normalized.
- tests/corpus/reference_export/corpus_capture_02.normalized.json - a second capture of the same protocol.
- examples/corpus_rule_v1.json - the first rule, too narrow on the command code.
- examples/corpus_rule_v2.json - the refined rule.

Start the window:

```
python -m src.ui.main_window \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --rule examples/corpus_rule_v1.json
```

The window opens whatever capture and rule you pass on the command line or through File, Open capture and File, Open rule. It also ships a small example (a capture, a rule and a journal under src/ui/samples/), reachable through File, Open demo (Ctrl+D); the walkthrough below uses the reference corpus so the numbers can be checked against docs/METRICS.md.

Before the walkthrough, confirm the whole suite is green:

```
QT_QPA_PLATFORM=offscreen pytest tests/ -q
```

Step 1 - open the capture (1 min)

Do: File, Open capture, choose corpus_capture_01.normalized.json. The same dialog also accepts a raw .pcap/.pcapng, which the window normalizes in memory on open; File, Import PCAP… reads a raw capture and additionally writes its *.normalized.json next to the source before opening it.

See: three sessions in the tree, each with its two directions and byte counts. The status bar shows the capture id and the diagnostics.

Say: the tool opens a normalized capture directly, and normalizes a raw pcap/pcapng on the fly. Either way each range remembers the packet it came from.

Step 2 - provenance of a byte (1 min)

Do: select session `s2`, direction `A_to_B`. Move the pointer over the first bytes.

See: the status bar shows the session, the direction, the offset, and the packet index, sequence number and time of the byte.

Say: a byte is tied to the packet it came from. A claim about a byte can always be checked against the capture.

Step 3 - apply the first rule (2 min)

Do: File, Open rule, `corpus_rule_v1.json`. Then Rule, Apply to current direction, or press F5.

See: the Interpretation tab lists messages with their status. The counterexample tab fills up; the status line reads the rule version and the counts.

Say: the rule frames every message from one description; the messages are not enumerated by hand. Green is matched, red is a counterexample.

Step 4 - a counterexample and its bytes (2 min)

Do: open the Counterexamples tab, select the first entry.

See: the hex view scrolls to the message and the offending bytes are highlighted; the status bar shows their packet and sequence number.

Say: this request carries command `2`, but rule v1 only allowed `1`. The counterexample is tied to real bytes, not to a description of them.

Step 4b - alternative readings (1 min)

Do: Rule, Apply to whole capture, or press F6, keeping rule v1. Then open the Hypotheses tab.

See: one row per candidate reading of each `hypothesis` field, with its support, its contradiction count and its score. For `command`, `entropy_enum` and `low_cardinality` top the list at 1.00 (140 support, 0 contradiction), while `constant` - the reading rule v1 declares - sits at 0.43 (60 support, 80 contradiction).

Say: before widening the rule, the field is put to the alternatives engine over every message the rule applies to. The reading the rule already declares is only one candidate, and on this capture most messages contradict it. That is the evidence behind the refinement, not a guess. Selecting a row jumps the hex view to a message that supports or contradicts it.

Step 5 - refine the rule (2 min)

Do: File, Open rule, `corpus_rule_v2.json`. Apply again to `s2`, `A_to_B`.

See: the counterexample list is empty; every message is matched.

Say: the refinement widened the allowed command codes to `1`, `2` and `3`. The structure of the rule did not change. The earlier result stays on disk with its old version and is marked outdated.

Step 6 - verify over the whole capture (1 min)

Do: Rule, Apply to whole capture, or press F6.

See: the status line reports the counts over every direction: `matched=140, mismatched=0, ...`.

Say: the rule is checked over the whole corpus, not one message. Rule v1 gave 60 matched and 80 counterexamples; rule v2 gives 140 matched and none.

Step 7 - transfer to a second capture (1 min)

Do: File, Open capture, `corpus_capture_02.normalized.json`. Apply to whole capture with rule v2.

See: 40 matched, 0 counterexamples, 40 not applicable (the responses, which the rule does not scope).

Say: the rule was not fitted to this capture, and it still holds. That is evidence of generality within this protocol.

Step 8 - the applicability boundary (2 min)

Do: the synthetic capture is not shipped as a normalized export, because it is outside the reference corpus. Normalize it first, then open it:

```
python -m src.capture.cli --pcap tests/corpus/synthetic_live.pcapng \
  --out /tmp/synthetic_live.normalized.json
```

Then File, Open capture, `/tmp/synthetic_live.normalized.json`. Apply to whole capture with rule v2.

See: no message matches; the framing reports the stream as incomplete or not applicable, so the counterexample set is not empty and the match count is zero.

Say: a different capture uses a little-endian length, a transaction id and other command codes. The rule does not transfer. The tool reports the failure instead of hiding it. This is the boundary of the rule's applicability.

Step 9 - journal correlation (1 min)

Do: this correlation is shown by the investigation script rather than the window. Run:

```
python -m scripts.run_reference_investigation
```

See: the journal section reports 140 of 140 requests correlated on the first capture and 40 of 40 on the second, with the decoded target equal to the journal parameter on every request.

Say: an independent record from outside the bytes agrees with the reading of the parameter field. It is evidence, not proof.

Step 10 - report and portable project (1 min)

Do: Report, Show REPORT.md. Then Report, Export Markdown, save to a file.

See: the report in the panel and the exported file.

Say: the investigation is written up with its counterexamples, its versions and its limits. The project directory can be copied elsewhere and opened again.

The portable project is driven from the Project menu in the window, and from the command line when a script is preferred. To show the CLI path, run:

```
python -m src.project.cli pipeline \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --project /tmp/demo.madrigal --rule examples/corpus_rule_v2.json
python -m src.project.cli export --project /tmp/demo.madrigal --out /tmp/demo.zip
python -m src.project.cli import --in /tmp/demo.zip --target /tmp/demo.moved
```

See: the pipeline writes a project with the capture, the normalized JSON and the rendered report; the export and import round trip keeps the capture digest. The full command line walkthrough is `docs/demo_cli.md`.

Fallback - command line

If the window cannot run on the demonstration machine, the same result comes from the command line:

```
QT_QPA_PLATFORM=offscreen python -m src.protocol.cli apply \
  --rule examples/corpus_rule_v2.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --out /tmp/result.json

QT_QPA_PLATFORM=offscreen python -m src.protocol.cli verify \
  --rule examples/corpus_rule_v2.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json

QT_QPA_PLATFORM=offscreen python -m scripts.run_reference_investigation
```

The `apply` command writes a result that validates against `docs/schemas/result.schema.json`. The `verify` command prints the counts and the counterexamples. The investigation script regenerates the full write-up.

Timing

step	minutes
1 open capture	1
2 provenance	1
3 apply v1	2
4 counterexample	2
4b alternatives	1
5 refine	2
6 whole capture	1
7 transfer	1
8 boundary	2
9 journal	1
10 report	1
total	15